МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

Учреждение образования «БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет Информационных технологий
Кафедра Информационные системы и технологии
Специальность 1–40 01 01 Программное обеспечение информационных технологий

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА К КУРСОВОЙ РАБОТЕ НА ТЕМУ:

«Web-приложение «Онлайн-курсы по русскому языку»

Выполнил студент Авдеева Вера Дмитриевна
(Ф.И.О.)

Руководитель работы ассист. Нистюк О.А.
(учен. степень, звание, должность, Ф.И.О., подпись)

И.о. зав. кафедрой ст. преп. Блинова Е.А.
(учен. степень, звание, должность, Ф.И.О., подпись)

Курсовая работа защищена с оценкой

Минск 2024

Содержание

Введение ... 3
1 Постановка задачи ... 4
1.1 Аналитический обзор аналогов по теме ... 4
1.1.1 Сервис Coursera ... 4
1.1.2 Сервис Грамотей ... 5
1.2 Определение основных требований к web-приложению ... 6
1.2.1 Определение функциональных требований ... 6
1.2.2 Определение нефункциональных требований ... 7
1.3 Вывод по разделу ... 7
2 Проектирование web-приложения. ... 9
2.1 Архитектура web-приложения ... 9
2.2 Проектирование структуры базы данных ... 10
3 Разработка web-приложения ... 13
3.1 Разработка серверной части ... 13
3.1.1 Модели ... 13
3.1.2 Контроллеры ... 14
3.1.3 Routes ... 15
3.1.4 Вспомогательные компоненты ... 16
3.2 Разработка клиентской части ... 17
4 Тестирование web-приложения ... 20
5 Руководство пользователя ... 23
Заключение ... 32
Список используемых источников ... 33
ПРИЛОЖЕНИЕ А ... 34
ПРИЛОЖЕНИЕ Б ... 35
ПРИЛОЖЕНИЕ В ... 36
ПРИЛОЖЕНИЕ Г ... 37
ПРИЛОЖЕНИЕ Д ... 39

Введение

В современном мире онлайн-образование становится все более популярным и доступным. В рамках этой тенденции создание web-приложения для изучения русского языка онлайн представляет собой актуальную и значимую задачу. Важно разработать удобное и функциональное приложение, которое бы удовлетворяло потребности разнообразных пользователей.

Целью данного проекта является создание web-приложения «Онлайн-курсы по русскому языку», которое предоставит пользователям возможность эффективно изучать русский язык в удобном онлайн формате. Приложение будет обладать широким спектром функциональности, включая регистрацию и авторизацию, поддержку разных ролей пользователей, возможность управления учебным контентом, просмотр каталога курсов, запись на курсы, прохождение тестов для закрепления знаний, а также просмотр личной статистики и поиск курсов по различным критериям.

Для достижения поставленной цели необходимо разработать web-приложение с использованием асинхронного программирования на платформе Node.js. с использованием TypeScript, а также выполнить следующие требования:

− Приложение должно обеспечивать возможность регистрации и авторизации пользователей, гарантируя безопасность и конфиденциальность их данных.

− Система должна предоставлять возможность проведения тестов для закрепления знаний, включая автоматическую проверку результатов и выдачу обратной связи, пользователи в свою очередь должны иметь возможность просматривать личную статистику, включающую в себя результаты тестов и другую сопутствующую информацию.

− Приложение должно предоставлять возможность добавления, удаления и редактирования теоретических материалов по темам курсов для администраторов.

− Пользователи должны иметь доступ к каталогу курсов, где они смогут ознакомиться с описанием курсов, их содержанием и требованиями, также приложение должно позволять пользователям записываться на выбранные курсы, обеспечивая удобный процесс выбора и регистрации.

1 Постановка задачи

1.1 Аналитический обзор аналогов по теме

Перед началом разработки веб-приложения важно провести анализ существующих аналогов, изучив их достоинства и недостатки. В процессе исследования удалось обнаружить информацию о нескольких аналогичных веб-приложениях, которые могут быть релевантными для анализа в контексте создания онлайн-курсов по русскому языку.

1.1.1 Сервис Coursera

Coursera – это всемирно известная образовательная платформа, которая предлагает онлайн-курсы, специализации и степени от ведущих университетов и компаний. Coursera стремится предоставить доступ к высококачественному образованию людям по всему миру, независимо от их физического местоположения или социального статуса.

Онлайн-платформа Coursera предлагает курсы по широкому спектру дисциплин, от гуманитарных наук и искусства до технических и естественных наук. Интерфейс представлен на рисунке 1.1.

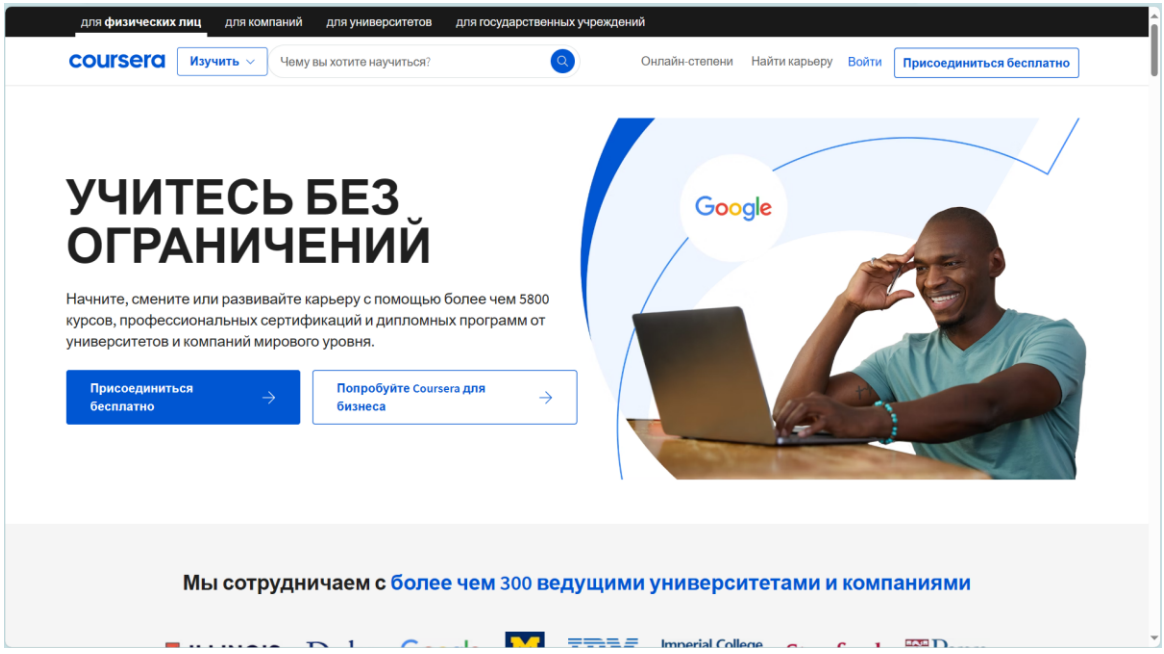


Рисунок 1.1 – Интерфейс платформы «Coursera»

Ключевые особенности Coursera включают в себя:

− Широкий выбор курсов: Пользователи могут изучать различные предметы, начиная от основ программирования и заканчивая философией и дизайном.

− Гибкий график обучения: Курсы Coursera позволяют учиться в удобное для пользователя время и в удобном темпе, что делает образование доступным даже для занятых людей.

– Сертификаты и степени: По завершении курсов пользователи могут получить сертификаты, а также заработать степени, которые признаются в индустрии и академическом сообществе.

– Интерактивное обучение: Курсы включают видеолекции, самостоятельные тесты, проекты и задания, которые обеспечивают глубокое погружение в предмет.

– Финансовая помощь и стипендии для тех, кто не может позволить себе оплату курсов.

– Многоязычный контент: Некоторые курсы и материалы доступны на нескольких языках, что делает платформу доступной для международной аудитории.

В целом, данный сервис предоставляет обширные возможности для онлайн-обучения. Его можно считать эталонным в данной сфере.

1.1.2 Сервис Грамотей

В качестве более родственного аналога можно рассмотреть платформу Грамотей.

Грамотей – это веб-приложение, предназначенное для изучения и практики русского языка, особенно в аспектах грамматики и орфографии. Это ресурс, который ориентирован на тех, кто стремится улучшить свои знания русского языка, в том числе для носителей языка, желающих закрепить и углубить свои навыки.

Грамотей предлагает разнообразные упражнения и тесты, которые охватывают широкий спектр тем от правил правописания до сложных грамматических конструкций. Платформа может быть особенно полезна для подготовки к экзаменам, таким как ЕГЭ и ОГЭ, а также для иностранцев, изучающих русский язык. Интерфейс приложения предоставлен на рисунке 1.2.

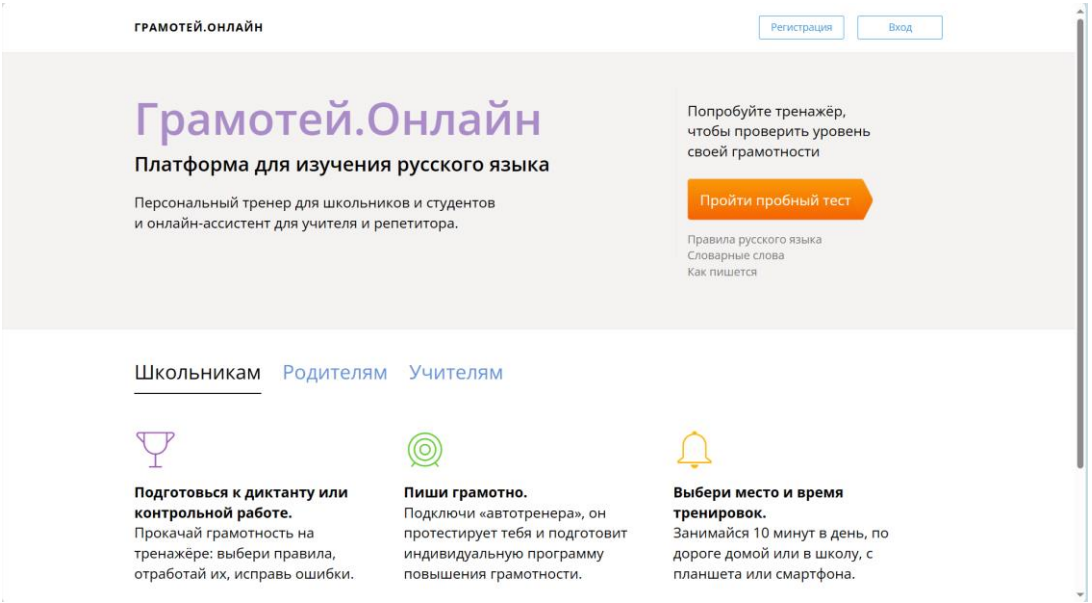


Рисунок 1.2 – Интерфейс приложения «Грамотей»

Ключевые особенности Грамотея включают в себя:

− Фокус на Грамматике: Грамотей уделяет особое внимание грамматическим аспектам русского языка, предоставляя подробные объяснения правил и исключений.

− Интерактивные Упражнения: Пользователи могут проверить свои знания, выполняя интерактивные задания, которые адаптируются под их уровень владения языком.

− Подготовка к Экзаменам: Платформа предлагает специализированные тренировочные тесты для тех, кто готовится к официальным экзаменам по русскому языку.

− Обратная Связь: После выполнения заданий пользователи получают обратную связь, которая помогает понять допущенные ошибки и улучшить знания.

− Доступность: Грамотей доступен бесплатно, что делает его отличным ресурсом для всех желающих улучшить свои знания русского языка.

− Поддержка Сообщества: Пользователи могут общаться на форумах и обмениваться знаниями, что способствует совместному обучению и поддержке.

Недостатки веб-приложения Грамотей:

− Ограниченный Контент: В отличие от более крупных платформ, Грамотей может предлагать меньше ресурсов и курсов по некоторым специальным или узким темам.

− Дизайн и Интерфейс: Интерфейс приложения может быть менее современным или интуитивно понятным по сравнению с более новыми образовательными платформами.

− Недостаток Мультимедийного Контента: Возможно, Грамотей предлагает меньше видеоматериалов и аудиоуроков, что может быть менее привлекательно для некоторых пользователей.

− Реклама: Бесплатная версия может включать рекламу, которая может отвлекать от процесса обучения.

В целом, Грамотей может быть ценным ресурсом для тех, кто ищет специализированные упражнения по русской грамматике и орфографии. Его преимущества делают его полезным инструментом для улучшения знаний русского языка, несмотря на некоторые недостатки.

1.2 Определение основных требований к web-приложению

Важным этапом разработки проекта является изучение требований. Требования к проекту могут быть как функциональными, то есть связанными с основными функциями проекта, так и нефункциональными, то есть связанными с качественными характеристиками проекта.

1.2.1 Определение функциональных требований

Функциональные требования описывают основные функции и возможности, которые веб-приложение должно предоставлять пользователям. Для web-приложения "Онлайн-курсы по русскому языку" были определены следующие функциональные требования:

– Регистрация и Авторизация: Веб-приложение должно предоставлять механизмы регистрации и авторизации для пользователей.

– Роли Пользователей: Система должна поддерживать различные уровни доступа и функциональности для администраторов и обычных пользователей.

– Управление Контентом: Администраторы должны иметь возможность добавлять, удалять и редактировать теоретические материалы и курсы.

– Каталог Курсов: Пользователи должны иметь доступ к просмотру каталога доступных курсов.

– Запись на Курс: Пользователи должны иметь возможность записываться на интересующие их курсы.

– Тестирование: Веб-приложение должно предоставлять тесты для закрепления знаний пользователей. Пользователи должны иметь возможность просматривать свою личную статистику.

– Поиск Курсов: Пользователи должны иметь возможность искать курсы по различным критериям.

Эти функциональные требования будут реализованы в рамках курсового проекта для обеспечения комплексного и эффективного обучения.

1.2.2 Определение нефункциональных требований

В качестве нефункциональных требований, можно выделить:

– безопасность: система должна обеспечивать безопасность данных и защиту от несанкционированного доступа, включая механизмы шифрования, аутентификации и авторизации пользователей;

– производительность: web-приложение должно обеспечивать быстрый и стабильный доступ к данным и курсам, что важно для комфортного обучения.;

– кроссплатформенность: web-приложение должно корректно функционировать в различных браузерах и на разных устройствах, включая мобильные;

Эти нефункциональные требования важны для создания надежного и удобного web-приложения, которое сможет удовлетворить потребности пользователей и обеспечить качественное обучение.

1.3 Вывод по разделу

В данном разделе был проведен аналитический обзор аналогов, включающий два конкретных примера: «Coursera» и «Грамотей». Это позволило определить ключевые функции и особенности, которые должны быть реализованы в новом веб-приложении для изучения русского языка, а также выявить потенциальные преимущества и возможные недостатки в сравнении с существующими решениями.

Сформулированная задача проекта чётко определяет его цель – создание многофункционального web-приложения, предоставляющего обширные возможности для глубокого изучения русского языка, включая регистрацию и авторизацию пользователей, управление курсами и контентом, а также поддержку различных ролей пользователей.

Были выделены функциональные требования, подробно описывающие функционал системы, включая регистрацию, авторизацию, управление контентом и курсами, запись на курсы, тестирование, статистику и поиск курсов. Нефункциональные требования обозначили критерии качества, такие как безопасность, производительность и кроссплатформенность, которые обеспечат надёжную и эффективную работу web-приложения.

Диаграмма вариантов использования представлена в приложении А.

В результате проведённого анализа была создана основа для дальнейшей разработки проекта, который будет учитывать как успешные практики аналогов, так и специфические требования к обучающему ресурсу по русскому языку. Это позволит реализовать web-приложение, которое отвечает современным требованиям пользователей и обеспечивает качественное образовательное взаимодействие.

2 Проектирование web-приложения.

2.1 Архитектура web-приложения

При разработке web-приложения для курсового проекта были разработаны две ключевые части: серверная и клиентская.

Серверная часть приложения разработана при помощи фреймворка Express.js. Были реализованы следующие составляющие:

Controllers: содержатся контроллеры, отвечающие за обработку запросов от клиента и взаимодействие с другими компонентами серверной части приложения. Каждый контроллер отвечает за определенный набор функциональных возможностей и выполняет соответствующие операции с данными.

Models: содержатся модели данных, которые используются для взаимодействия с базой данных. Каждая модель представляет определенный тип данных и определяет его структуру и методы доступа к данным. Это обеспечивает единообразное представление данных и упрощает их манипуляцию и использование в приложении.

Middleware: содержит промежуточное программное обеспечение, которое выполняет различные операции обработки запросов перед их передачей контроллерам. В данном курсовом проекте включает в себя аутентификацию пользователей, проверку прав доступа и обработчик ошибок, необходимый для эффективного управления возможными ошибками.

Services: содержит независимые сервисы, которые используются различными компонентами приложения для выполнения определенных задач.

Utils: содержатся вспомогательные утилиты и функции, которые используются в различных частях приложения. В данном web-приложении это функции для JWT-токенов, подключения базы данных, подключения Redis, который используется для кеширования некоторой информации, а также NodeMailer, используемый для отправки писем на почту пользователям

Mails: содержит шаблоны для отправки электронных писем. В ней хранятся предварительно созданные шаблоны писем, отправляемых пользователям при необходимости. Это позволяет легко настраивать и изменять содержание отправляемых писем, а также обеспечивает единообразие в оформлении уведомлений.

Routes: определены маршруты для обработки запросов клиента и их направление к соответствующим контроллерам. Это позволяет эффективно управлять маршрутизацией запросов и обеспечивает модульную структуру приложения.

Для реализации клиентской части был использован фреймворк Next.js. Одним из ключевых преимуществ Next.js является его поддержка статической генерации и предварительного рендеринга (SSG и SSR), что позволяет создавать быстрые и оптимизированные приложения. Кроме того, Next.js предоставляет интеграцию с маршрутизацией, асинхронной загрузкой данных, а также позволяет легко работать с внешними API и серверным состоянием приложения. Это упростило разработку и поддержку приложения. Благодаря модульной архитектуре Next.js и его экосистеме

плагинов и расширений, были интегрировать дополнительные функциональные возможности, такие как авторизация и кэширование.

Для хранения данных была выбрана база данных MongoDB. Решающим фактором для ее выбора стала гибкая схема данных. Поскольку MongoDB не требует строгой предварительной схемы, есть возможность легко добавлять новые поля или изменять структуру данных в процессе развития приложения, что упрощает и ускоряет процесс разработки.

Также в приложении используется WebSocket, позволяющий установить постоянное соединение между клиентом и сервером, что обеспечивает эффективную передачу данных в обоих направлениях без необходимости постоянных повторных запросов с клиента на сервер. В данном web-приложении используется для реализации системы уведомлений.

Диаграмма развертывания представлена в приложении Б

2.2 Проектирование структуры базы данных

Для реализации работы базы данных было создано 5 коллекций, состоящих из различных документов и объектов. Были созданы коллекции Courses, Orders, Users, Layouts и Notifacations.

Коллекция USERS хранит информацию о пользователях платформы. Состоит из следующих документов и объектов (таблица 2.1):

Таблица 2.1 –Документа и объекты коллекции USERS

Наименование	Описание	Тип
_id	Внутренний уникальный идентификатор пользователя	ObjectId
name	Имя пользователя	String
email	Электронная почта пользователя	String
password	Пароль пользователя	String
courses	Массив объектов-курсов, на которые записан пользователь	Array[Courses]
avatar	Объект аватара пользователя	Object

Коллекция COURSES содержит информацию о курсе. Состоит из документов и объектов (таблица 2.2):

Таблица 2.2 – Документы и объекты коллекции COURSES

Наименование	Описание	Тип
_id	Внутренний уникальный идентификатор курса, первичный ключ	ObjectId
name	Название курса	String
description	Краткое описание курса	String

Продолжение таблицы 2.2

categories	Категория, к которой относится курс	String
price	Цена курса конечная, с возможными скидками	Int
estimatedPrice	Цена курса изначальная	Int
thumbnail	Заставка для курса	Object
tags	Теги курса	String
level	Уровень подготовки, необходимый для курса	String
demoUrl	URL видео-заставки предварительной у курса	String
benefits	Массив, содержащий объекты с описанием того, что будет достигнуто после прохождения курса	Array[benefits]
prerequisites	Массив, содержащий объекты с описанием предварительных условий для прохождения курса	Array[prerequisites]
courseData	Массив объектов данных о курсе, где содержатся отдельные данные для каждого урока	Array[courseData]
ratings	Объект рейтинга курса	Object
purchased	Количество человек, записавшихся на курс	Int
reviews	Массив, содержащий объекты с отзывами пользователей	Array[reviews]

Коллекция ORDERS содержит информацию о заказах. Состоит из объектов и документов (таблица 2.3):

Таблица 2.3 – Документы и объекты коллекции ORDERS

Наименование	Описание	Тип
_id	Внутренний уникальный идентификатор заказа, первичный ключ	ObjectId
courseId	Идентификатор заказываемого курса	String
userId	Идентификатор заказчика	String
payment_info	Объект, содержащий обширную информацию о заказе	Object

Коллекция NOTIFICATIONS содержит информацию об уведомлениях. Состоит из документов (таблица 2.4):

Таблица 2.4 – Документы коллекции NOTIFICATIONS

Наименование	Описание	Тип
_id	Внутренний уникальный идентификатор уведомления, первичный ключ	ObjectId
title	Заголовок уведомления	String
message	Текст уведомления	String
status	Статус просмотра уведомления	String

Коллекция LAYOUTS содержит информацию о шаблонных частях для приложения. Состоит из документов и объектов (таблица 2.5):

Таблица 2.5 –Документы и объекты коллекции LAYOUTS

Наименование	Описание	Тип
_id	Внутренний уникальный идентификатор шаблона, первичный ключ	ObjectId
type	Тип шаблона	NUMBER(*)
faq	Массив объектов с шаблонами для faq	Array[faq]
categories	Массив объектов с шаблонами для категорий	Array[categories]
banner	Массив объектов с шаблонами для баннера	Array[banner]

Разработанная база данных включает в себя 5 коллекция. Каждая из этих коллекций была создана с учетом конкретных потребностей образовательной платформы, что обеспечивает оптимальное управление информацией о пользователях, курсах, заказах и так далее.

Диаграмма базы данных представлена в приложении В

Таким образом была успешно спроектирована архитектура приложения, содержащая в себе серверную, клиентскую части и базу данных

3 Разработка web-приложения

3.1 Разработка серверной части

Серверный компонент приложения заботится о реагировании на запросы от клиентов, взаимодействует с базой данных для получения и обновления информации, обеспечивает безопасную идентификацию и разграничение доступа пользователей, а также поддерживает работу WebSocket для обеспечения двусторонней связи в реальном времени.

Входной точкой приложения является файл app.ts, к которому подключен файлы server.ts и socketServer.ts, инициализирующие HTTPS- и WebSocket-сервера соответственно.

Для создания HTTPS-сервера при помощи утилиты mkcert были сгенерированы парой ключ и сертификат для localhost. Генерация пары представлена на рисунке 3.1.

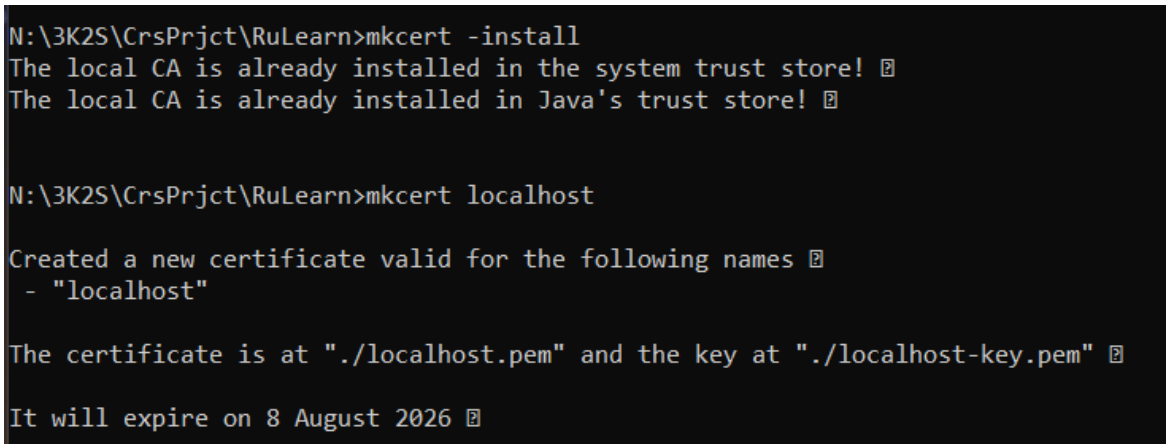


Рисунок 3.1 – Процесс генерации сертификата и ключа

Далее сгенерированная пара использовалась в качестве опций при инициализации сервера.

3.1.1 Модели

Для полноценной работы приложения необходимо было создать модели, которые соответствовали бы коллекциям базы данных, применялся подход Code-First. Было создано 5 основных моделей: UserModel, CourseModel, OrderModel, NotificationsModel и LayouytModel.

Пример создания в схеме User собственных функций а так же процесс хеширования пароля представлен на листинге 3.1.

```
// Hash Password before saving
userSchema.pre<IUser>("save", async function (next) {
    if (!this.isModified("password")) {
      next();
    }
    this.password = await bcrypt.hash(this.password, 10);
```

```
      next();
    });
  // sign access token
userSchema.methods.SignAccessToken = function () {
  return jwt.sign({ id: this._id }, process.env.ACCESS_TOKEN || "",
{
    expiresIn: "5m",
  });
};
// sign refresh token
userSchema.methods.SignRefreshToken = function () {
  return jwt.sign({ id: this._id }, process.env.REFRESH_TOKEN ||
"", {
    expiresIn: "3d",
  });
};
  // compare password
userSchema.methods.comparePassword = async function (
    enteredPassword: string
  ): Promise<boolean> {
    return await bcrypt.compare(enteredPassword, this.password);
  };
const userModel: Model<IUser> = mongoose.model("User", userSchema);
export default userModel;
```

Листинг 3.1 – Часть содержимого файла user.model.ts

Для усиленной обработки ошибок помимо валидации на верхних слоях приложения, была также предусмотрена валидация на уровне документов базы данных.

Аналогично были реализованы и остальные модели. Моделью с наиболее обширным количеством зависимостей и схем является CourseModel. Листинг создания данной модели приведен в приложении Г.

3.1.2 Контроллеры

Для обработки запросов от клиентов было создано несколько отдельных файлов, отвечающих за конкретные области запрашиваемого. Были созданы userController, courseController, orderController, notificationsController.

В файле user.controller.ts содержится обработка всех запросов, связанных главным образом с пользователем.

Пример реализации авторизации представлен на листинге 3.2

```
export const loginUser = CatchAsyncError(
  async (req: Request, res: Response, next: NextFunction) => {
    console.log('loginuser')
    try {
      const { email, password } = req.body as ILoginRequest;
      if (!email || !password) {
        return next(new ErrorHandler("Please enter email and
password", 400));
```

```
      }
      const user = await userModel.findOne({ email
}).select("+password");
      if (!user) {
        return next(new ErrorHandler("Invalid email or password",
400));
      }
      const isPasswordMatch = await user.comparePassword(password);
      if (!isPasswordMatch) {
        return next(new ErrorHandler("Invalid email or password",
400));
      }
      sendToken(user, 200, res);
    } catch (error: any) {
      return next(new ErrorHandler(error.message, 400));
    }
  }
);
```

Листинг 3.2 – Часть содержимого файла user.controller.ts

Аналогичным образом разработаны остальные котроллеры, содержащие необходимые функциональные возможности. OrdersController использует в функции для создания платежа стороннее платежное API Stripe. Листинг содержимого файла orders.controller.ts приведен в приложении Д.

3.1.3 Routes

Для обеспечения качественной маршрутизацией запросов были отдельно созданы необходимые UserRoute, CourseRoute, OrderRoute, NotificationRoute.

Реализация маршрутизации для Users представлена на листинге 3.3

```
import express from "express";
import{registrationUser,activateUser,loginUser,logoutUser,updateAcce
ssToken,getUserInfo,socialAuth,updateUserInfo,updatePassword,updateP
rofilePicture,getAllUsers,updateUserRole,deleteUser} from
"../controllers/user.controller";
  import { isAutheticated, authorizeRoles } from
"../middleware/auth";
  const userRouter = express.Router();
  userRouter.post("/registration", registrationUser);
  userRouter.post("/activate-user", activateUser);
  userRouter.post("/login", loginUser);
  userRouter.get("/logout",isAutheticated, logoutUser);
  userRouter.get("/refresh", updateAccessToken);
  userRouter.get("/me",  isAutheticated, getUserInfo);
  userRouter.post("/social-auth", socialAuth);
  userRouter.put("/update-user-info",  isAutheticated,
updateUserInfo);
  userRouter.put("/update-user-password", isAutheticated,
updatePassword);
  userRouter.put("/update-user-avatar", isAutheticated,
```

```
updateProfilePicture);
  userRouter.get("/get-users", isAutheticated,
authorizeRoles("admin"), getAllUsers);
  userRouter.put("/update-user",  isAutheticated,
authorizeRoles("admin"), updateUserRole);
  userRouter.delete("/delete-user/:id",  isAutheticated,
authorizeRoles("admin"), deleteUser);
  export default userRouter;
```

Листинг 3.3 – Содержимое файла user.route.ts

Затем созданные роуты использовались в основном файле app.ts. Пример использования представлен на листинге 3.4.

```
 app.use( "/api/v1", userRouter, orderRouter,
    courseRouter,
    notificationRouter,
    analitycsRouter,
    layoutRouter);
```

Листинг 3.4 – Часть содержимого файла app.ts

3.1.4 Вспомогательные компоненты

Для улучшенной и корректной работы приложения были реализованы некоторые вспомогательные компоненты.

Первый из них – Middleware. Это прослойка, обрабатывающая некоторым образом запросы перед передачей контроллеру. В данном проекте используется для проверки аутентификации и валидации ролей пользователя, а также для более эффективной обработки ошибок.

Пример валидации пользовательских ролей представлен на листинге 3.5.

```
  // validate user role
export const authorizeRoles = (...roles: string[]) => {
    return (req: Request, res: Response, next: NextFunction) => {
      if (!roles.includes(req.user?.role || "")) {
        return next(
          new ErrorHandler(
            `Role: ${req.user?.role} is not allowed to access this
resource`,
          403)
        );
      }
      next();
    };
  };
```

Листинг 3.5 – реализация валидации пользовательских ролей

Следующим вспомогательным компонентом были utils. Здесь содержатся файлы подключения к базе данных MongoDB с использованием пакета Mongoose.

Для кеширования некоторых данных приложения, чтобы обеспечить быстродействие и уменьшить нагрузку на сервер, использовался Redis из облачной платформы Upstash.

Для обеспечения авторизации использовался механизм JWT-токенов. Пример реализации вспомогательной работы JWT представлен на листинге 3.6

```
export const accessTokenOptions: ITokenOptions = {
  expires: new Date(Date.now() + accessTokenExpire * 60  * 60 *
1000),
  maxAge: accessTokenExpire * 60 * 60 * 1000,
  httpOnly: true,
  sameSite: "none",
  secure:true,
};
export const refreshTokenOptions: ITokenOptions = {
  expires: new Date(Date.now() + refreshTokenExpire * 24 * 60 * 60
* 1000),
  maxAge: refreshTokenExpire * 24 * 60 * 60 * 1000,
  httpOnly: true,
  sameSite: "none",
  secure: true,
};
export const sendToken = (user: IUser, statusCode: number, res:
Response) => {
  const accessToken = user.SignAccessToken();
  const refreshToken = user.SignRefreshToken();
  redis.set(user._id, JSON.stringify(user) as any,);
  res.cookie("access_token", accessToken, accessTokenOptions);
  res.cookie("refresh_token", refreshToken, refreshTokenOptions);
  res.status(statusCode).json({
    success: true,
    user,
    accessToken,
  });
};
```

Листинг 3.6 – Часть содержимого файла JWT.ts

Для обеспечения информирования пользователей, а так же для первичной регистрации и отправки активационного кода использовался пакет NodeMailer и была создана отдельная папка Mails с необходимыми шаблонами сообщений.

Сторонними сервисами, используемыми в приложении, являются Cloudinary – сервис, позволяющий облачно хранить изображения, VdoCipher – сервис для защищенного хранения видео, Stripe – реальная платежная система, в проекте используется для имитации платежей.

3.2 Разработка клиентской части

Клиентская часть отвечает за внешний интерфейс для взаимодействия пользователя с сервером. Приложение написано с использованием Next.js

В папке app хранятся все необходимые компоненты, хуки, стили и сущности приложения.

Для взаимодействия с сервером и отправки запросов использовался Redux и Redux Mutation. Пример основного файла apiSlice, использующегося для реализации других частей api, представлен на листинге 3.7

```
import { createApi, fetchBaseQuery } from
"@reduxjs/toolkit/query/react";
import { userLoggedIn } from "../auth/authSlice";
export const apiSlice = createApi({
    reducerPath: "api",
    baseQuery: fetchBaseQuery({
      baseUrl: process.env.NEXT_PUBLIC_SERVER_URI,}),
    endpoints: (builder) => ({
      refreshToken: builder.query({
        query: (data) => ({
          url: "refresh",
          method: "GET",
          credentials: "include" as const,}),}),
      loadUser: builder.query({
        query: (data) => ({
          url: "me",
          method: "GET",
          credentials: "include" as const,}),
        async onQueryStarted(arg, { queryFulfilled, dispatch }) {
          try {
            const result = await queryFulfilled;
            dispatch(
              userLoggedIn({
                accessToken: result.data.accessToken,
                user: result.data.user,})
            );
          } catch (error: any) {console.log(error);}
        },
      }),
    }),
  });
  export const { useRefreshTokenQuery, useLoadUserQuery } =
apiSlice;
```

Листинг 3.7 –Содержимое файла apiSlice.ts

Основным шаблоном, внутри которого строится все приложения является layout.js. В нем все дочерние компоненты оборачиваются во вспомогательные, такие как SessionProvider, ThemeProvider и другие. Пример файла приведен в листинге 3.8

```
"use client"
export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
```

```
}) {
  return (
    <html lang="en" suppressHydrationWarning={true}>
      <body
        className={`${poppins.variable} ${josefin.variable} !bg-
white bg-no-repeat dark:bg-gradient-to-b dark:from-gray-900
dark:to-black duration-300`}
      >
        <Providers>
          <SessionProvider>
            <ThemeProvider attribute="class" defaultTheme="system"
enableSystem>
              <Custom>
                <div>{children}</div>
              </Custom>
              <Toaster position="top-center" reverseOrder={false}/>
            </ThemeProvider>
          </SessionProvider>
        </Providers>
      </body>
    </html>
  );}
const Custom: FC<{ children: React.ReactNode }> = ({ children }) =>
{
  const { isLoading } = useLoadUserQuery({});
  useEffect(() => {
    socketId.on("connection", () => {});
  }, []);
  return <div>{isLoading ? <Loader /> : <div>{children}
</div>}</div>;
};
```

Листинг 3.8 –Содержимое файла layout.tsx

Также имеются отдельные директории для сертификатов, чтобы обеспечить работу на HTTPS, директория public для каких-либо необходимых статических файлов, а также директория utils, которая содержит файлы, функционально дополняющие приложение.

4 Тестирование web-приложения

Тестирование будет происходить на клиентской части приложения.

Для того, чтобы пользователю получить доступ ко всем функциям приложения, ему необходимо авторизоваться. На верхнем уровне на стороне клиента сделана валидация формы до отправки запроса при помощи пакетов Yup и Formik.

При попытке зайти и отправить пустые данные форма приобретает вид, как на рисунке 4.1.

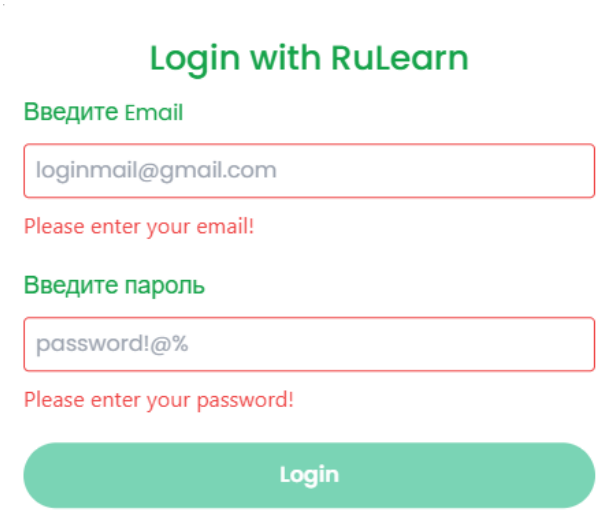


Рисунок 4.1 – Окно логина при попытке отправки пустых данных

При попытке ввода невалидных данных, которые не соответствуют ограничениям, также присутствует обработка в виде всплывающего окна, представленная на рисунке 4.2.

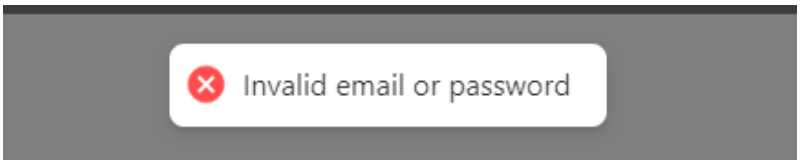


Рисунок 4.2 – Модальное окно с информацией об ошибке

В случае ввода существующих данных и успешной авторизации по ним появляется модальное окно, которое представлено на рисунке 4.3.



Рисунок 4.3 – Успешная авторизация

Аналогичная ситуация происходит при неверных попытках создания аккаунта при регистрации.

При попытке создания аккаунта с отсутствием данных, форма приобретает вид, представленный как на рисунке 4.4.

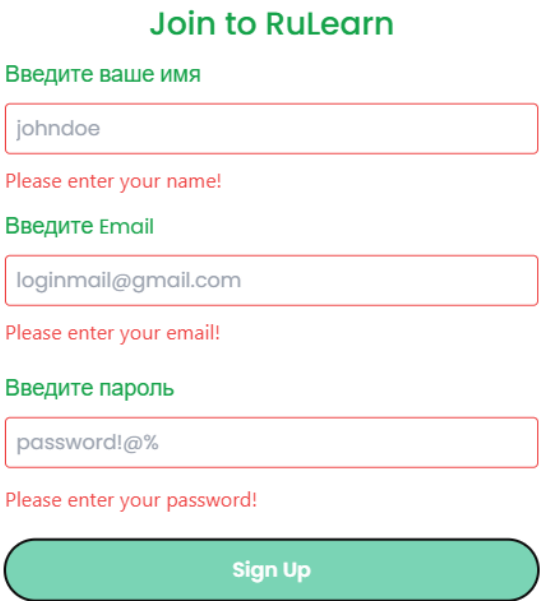


Рисунок 4.4 – Окно регистрации при попытке отправки пустых данных

При попытке зарегистрироваться по уже существующим данным появляется модальное окно, изображенное на рисунке 4.5, с соответствующей ошибкой.



Рисунок 4.5 – Модальное окно при попытке регистрации с существующей почтой

Затем, при вводе валидных данных при регистрации, на почту приходит письмо с активационным кодом, а форма приобретает вид, представленный на рисунке 4.6

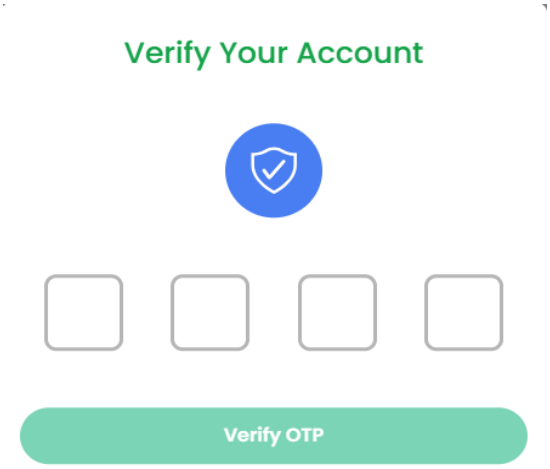


Рисунок 4.6 – Окно ввода кода для активации аккаунта

В случае ввода валидного активационного кода, аккаунт будет успешно создан.

Следующей была протестирована возможность создания курса администратором. При попытке отправить форму с хотя бы одним незаполненным полем, возникает предупреждение, представленное на рисунке 4.7.

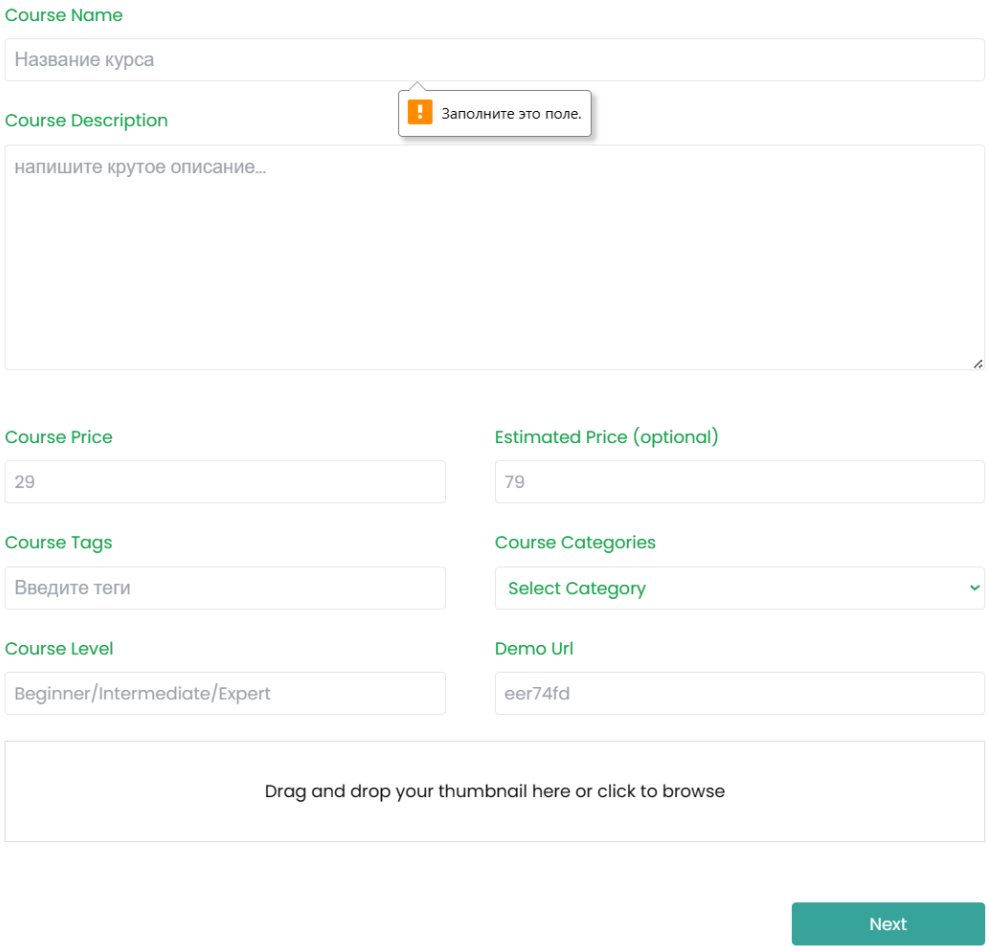


Рисунок 4.7 – ошибка при незаполнении полей

Окно, появляющееся при попытке отправки пустых данных на следующем этапе изображено на рисунке 4.8.

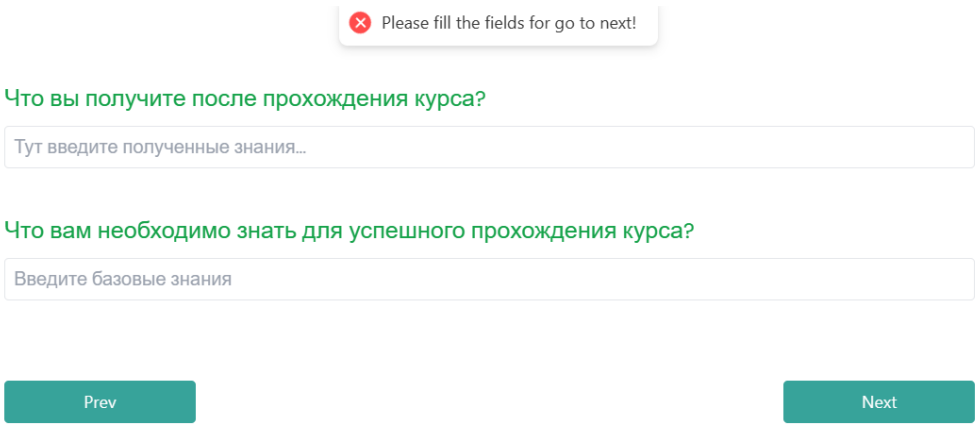


Рисунок 4.8 – ошибка при незаполнении полей

Таким образом, были протестированы основные функциональные моменты.

5 Руководство пользователя

При запуске приложения пользователь попадает на главную страницу, которая изображена на рисунке 5.1.

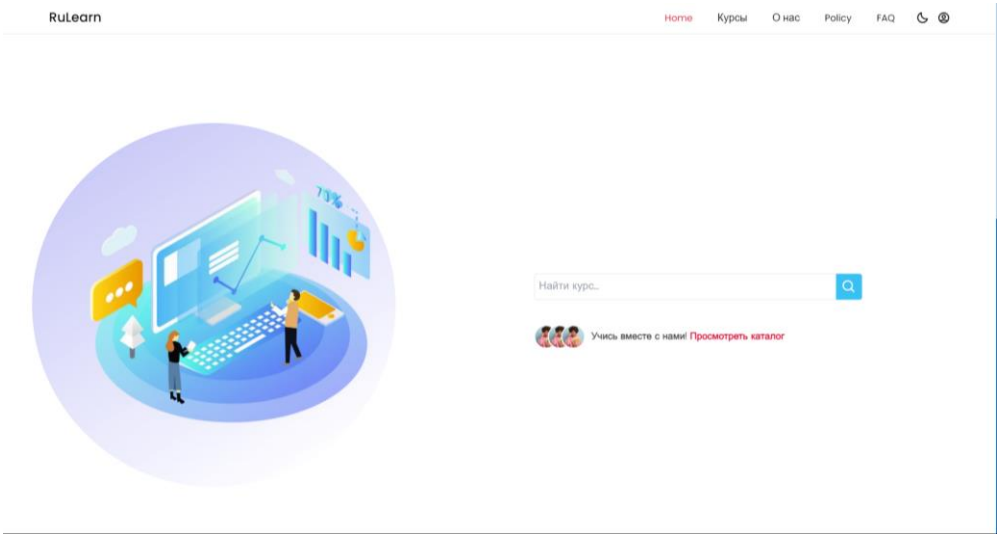


Рисунок 5.1 – главная страница приложения

Для того, чтобы полноценно использовать все возможности web-приложения, пользователю необходимо авторизоваться либо зарегистрироваться. При нажатии на иконку аватара появится всплывающее окно с предложением войти или зарегистрироваться. Окно, появляющееся перед пользователем, у которого уже есть аккаунт, изображено на рисунке 5.2.

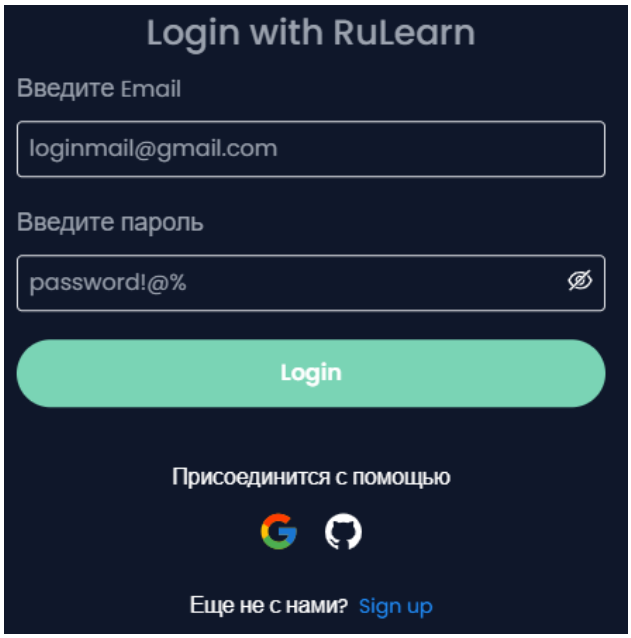


Рисунок 5.2 – Окно логина

В случае, если пользователь еще не имеет аккаунта, у него есть возможность его создать, нажав на кнопку Sign Up. Вторая возможность войти в приложение –

использовать Google- либо Github-авторизацию. Окно, появляющееся при регистрации приведено на рисунке 5.3.

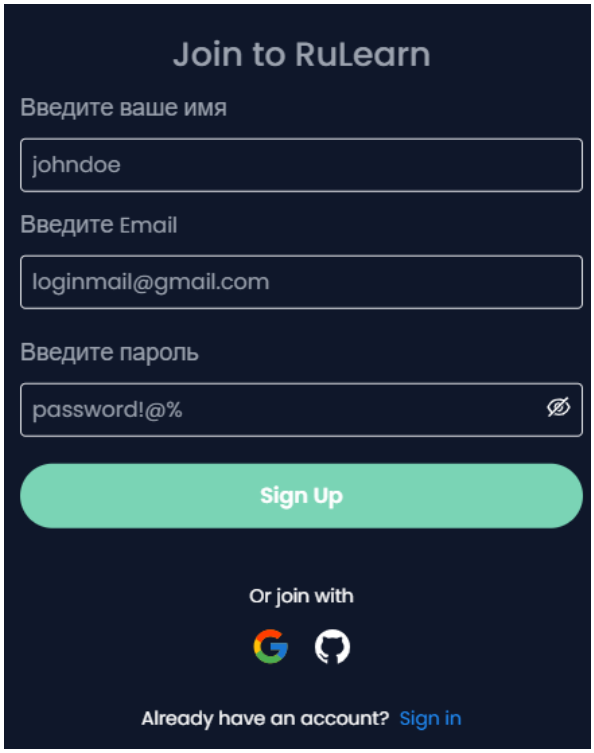


Рисунок 5.3 – Окно регистрации

Авторизованный пользователь может просмотреть список курсов, перейдя по вкладке Курсы. Результат представлен на рисунке 5.4.

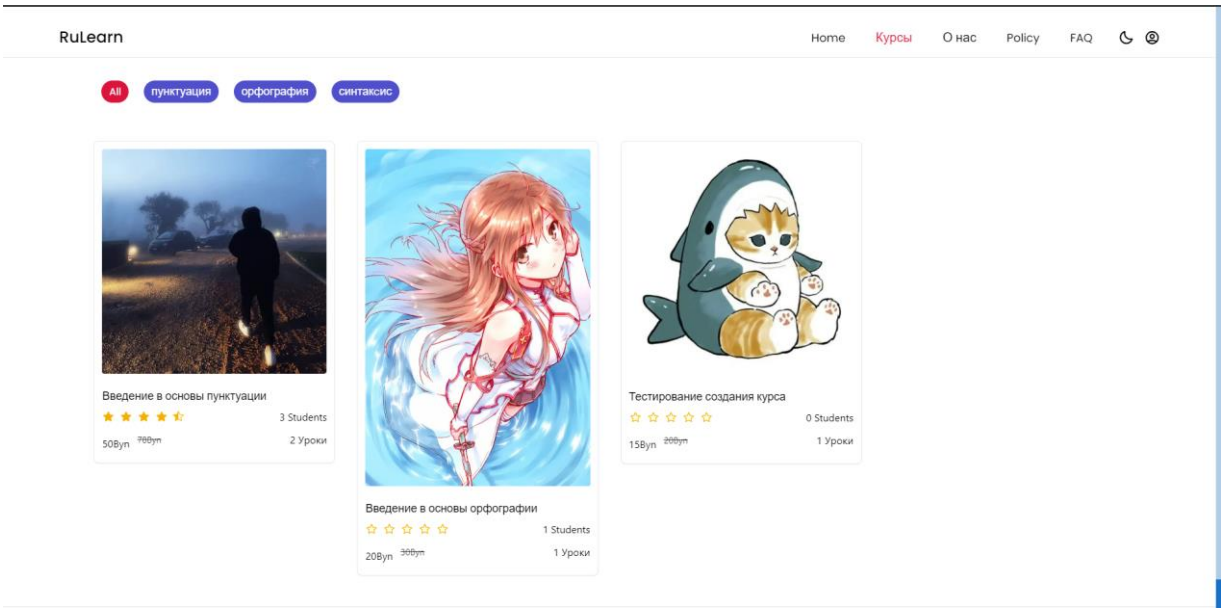


Рисунок 5.4 – страница Курсы

Для того, чтобы получить информацию о курсах, ему необходимо нажать на любой его интересующий курс.

Перед ним появится страница с общей информацией. Страница, которая ему откроется, изображена на рисунке 5.5.

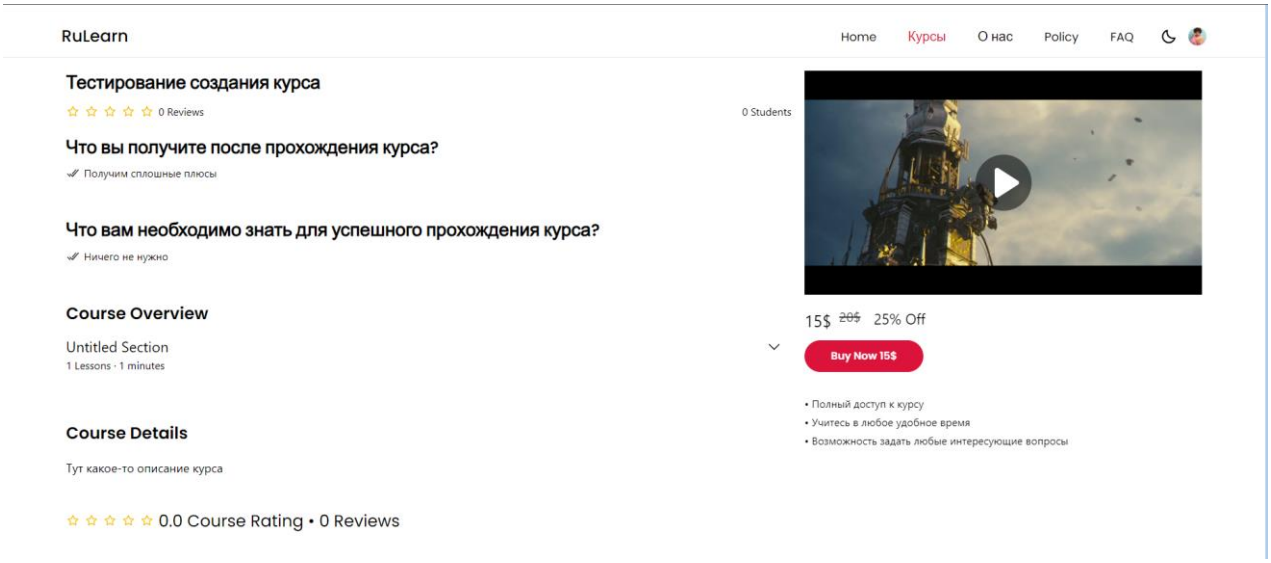


Рисунок 5.5 – предварительная информация о курсе

В случае, если пользователя заинтересовал курс, он может его купить. Для этого ему необходимо нажать на кнопку Buy. Перед пользователем возникнет форма, представленная на рисунке 5.6.

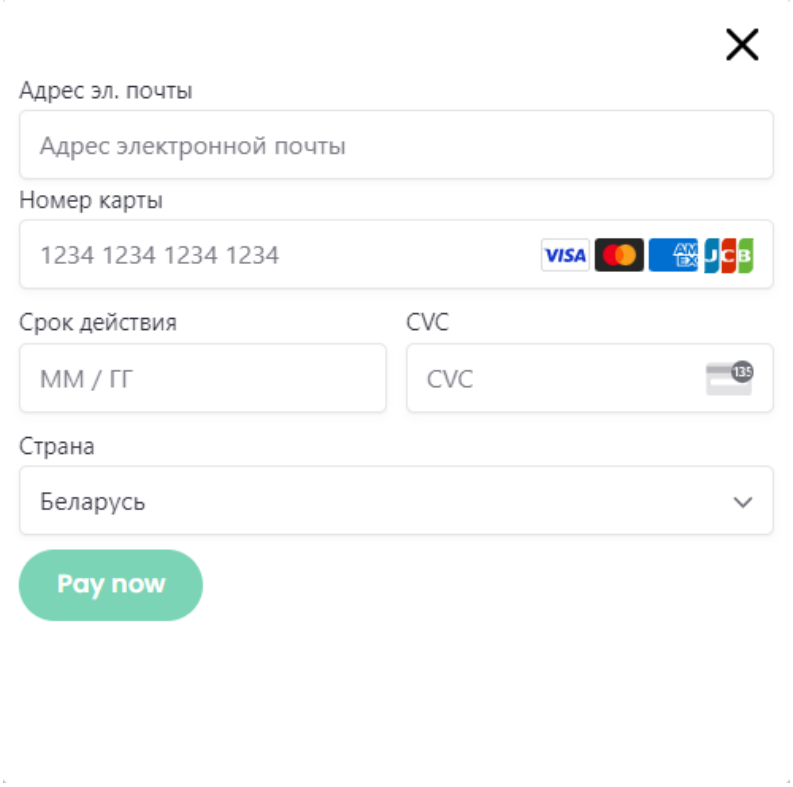


Рисунок 5.6 – Форма для оформления платежа

В случае успешного оформления оплаты, пользователя должно переадресовать на страницу, которая содержит полную информацию и данные

курса, где он может просмотреть интересующие его уроки. Появляющаяся страница представлена на рисунке 5.7.

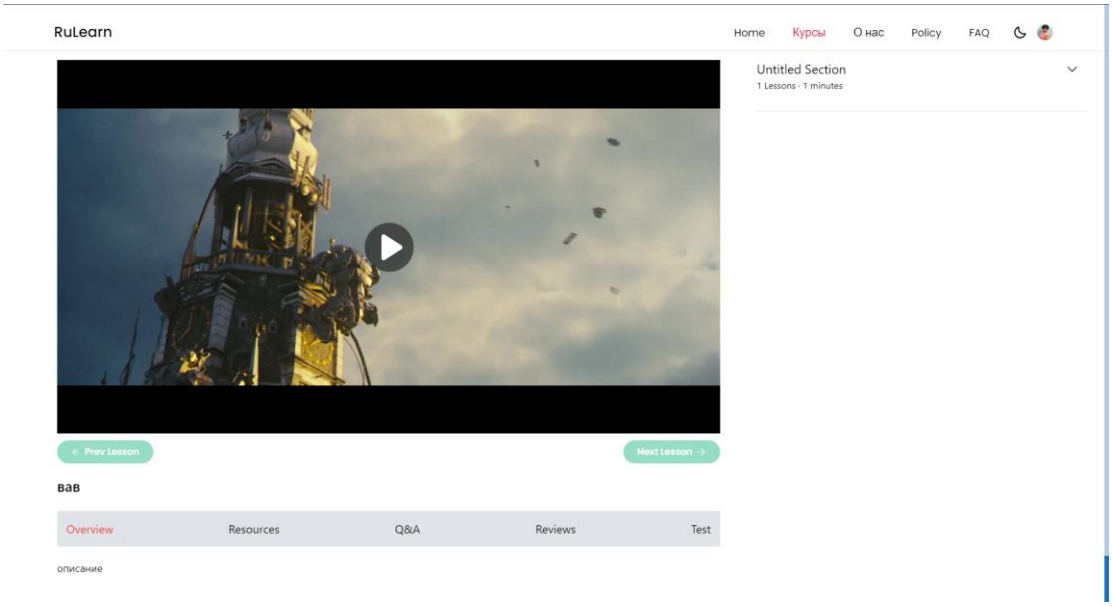


Рисунок 5.8 – Страница купленного курса

На странице курса пользователь имеет возможность перейти во вкладку Resources, где расположены ссылки на какой-либо дополнительный материал. Результат перехода изображен на рисунке 5.8.

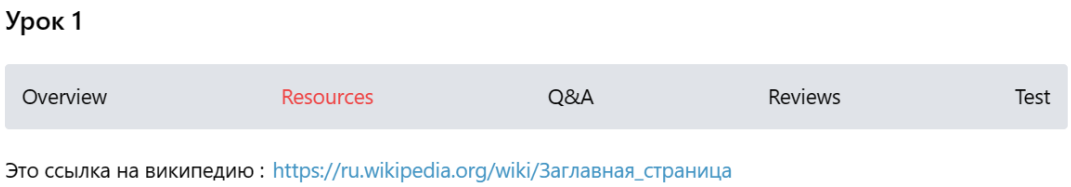


Рисунок 5.8 – Вкладка Resources

У пользователя есть возможность перейти во вкладку Q&A, где он может задать любой интересущий его вопрос. Результат перехода изображен на рисунке 5.9.

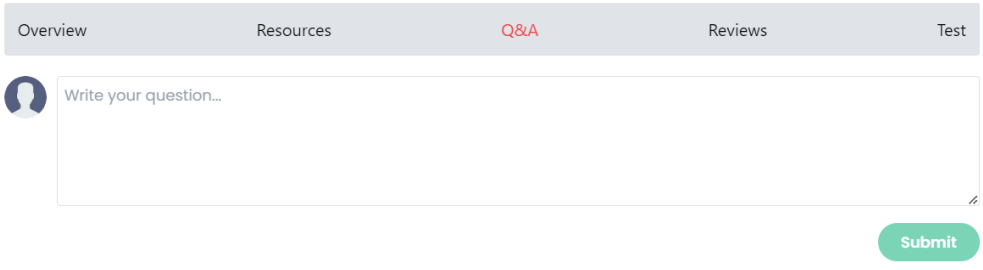


Рисунок 5.9 – Вкладка Q&A

У пользователя есть возможность перейти во вкладку Test, где ему предоставляется возможность закрепить знания, полученные после просмотра урока

путем прохождения тестовых вопросов. Результат перехода изображен на рисунке 5.10.

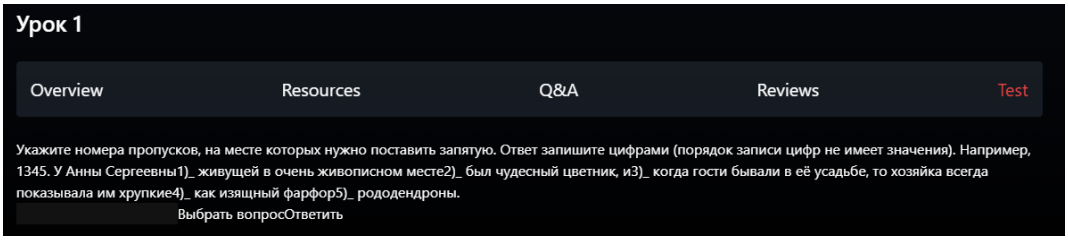


Рисунок 5.10 – Вкладка Test

Для перехода в личный профиль, пользователю необходимо нажать на иконку аватара в правом верхнем углу. Результат перехода изображен на рисунке 5.11.

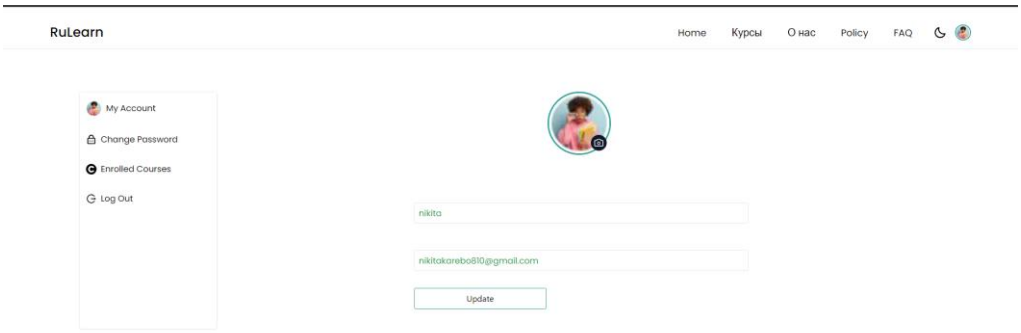


Рисунок 5.11 – Профиль пользователя

Там ему предоставляется возможность изменить свой никнейм и аватар. Смена электронной почты запрещена. Для изменения пароля, пользователю необходимо нажать на вкладку Change Password. Результат перехода изображен на рисунке 5.12.

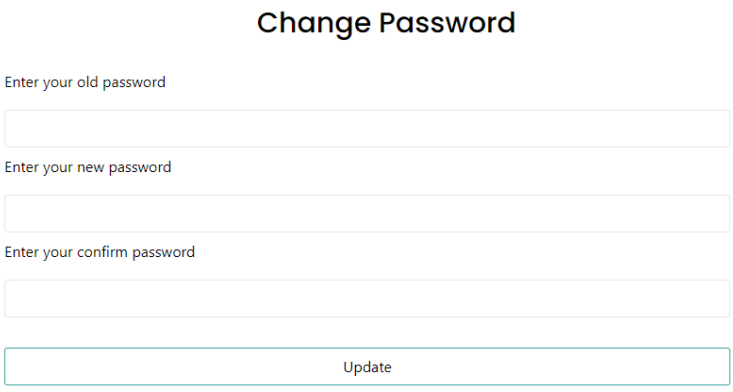


Рисунок 5.12 – Окно смены пароля

Также у пользователя в личном кабинете есть возможность просмотра всех курсов, на которые он записан на данный момент. Для просмотра этой информации

ему необходимо осуществить переход на вкладку под названием Enrolled Courses. Результат перехода на данную вкладку изображен на рисунке 5.13.

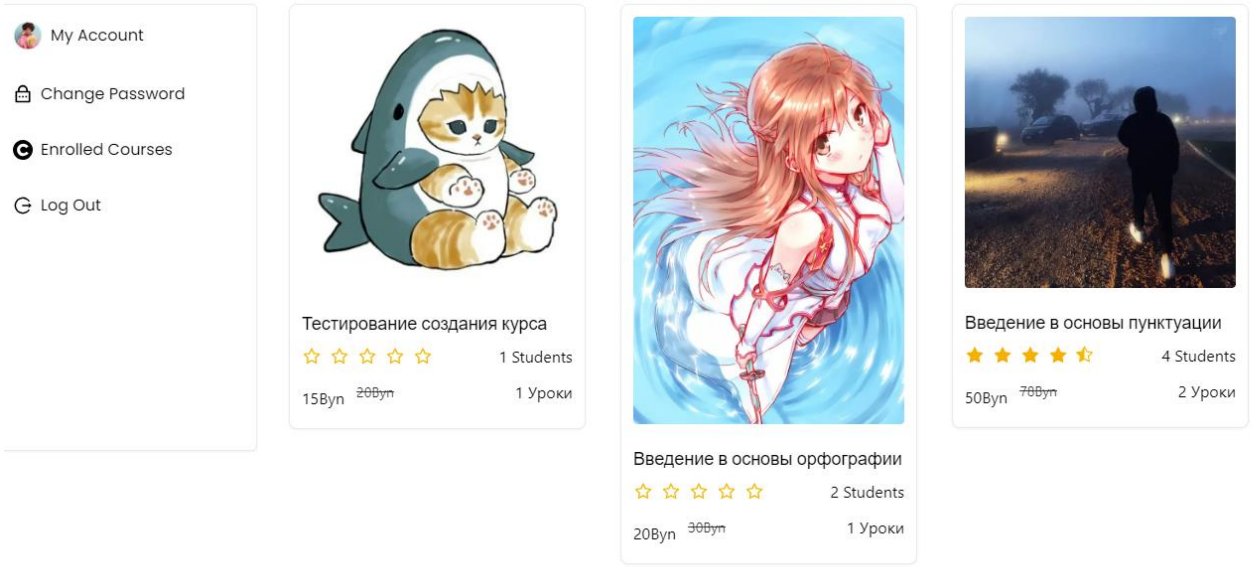


Рисунок 5.13 – Вкладка Enrolled Courses

У пользователя есть возможность перейти во вкладку Reviews, где он может оставить отзыв о курсе. Результат перехода изображен на рисунке 5.14.

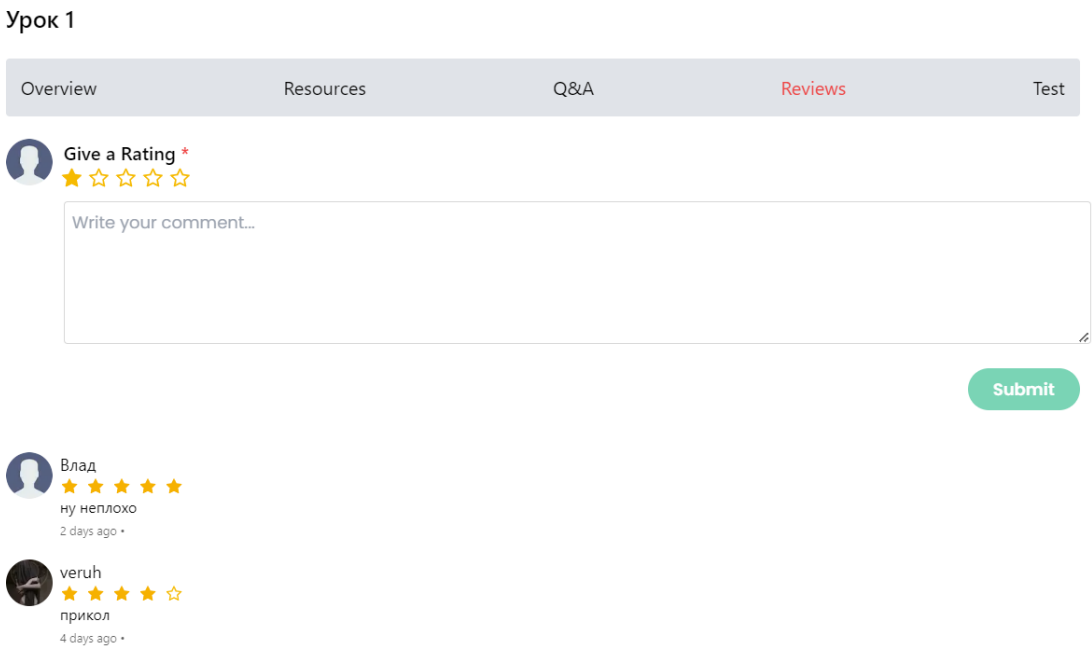


Рисунок 5.14 – Вкладка Reviews

Для более удобного доступа к курсам в каталоге в приложении реализован поиск. Для того, чтобы найти курс по названию, пользователю необходимо на главной станице ввести в поисковую строку название желаемого курса и нажать кнопку поиска. После этого произойдет переход на страницу каталога, где будут

показаны все курсы, подходящие под условия поиска. Страница с поиском представлена на рисунке 5.15.

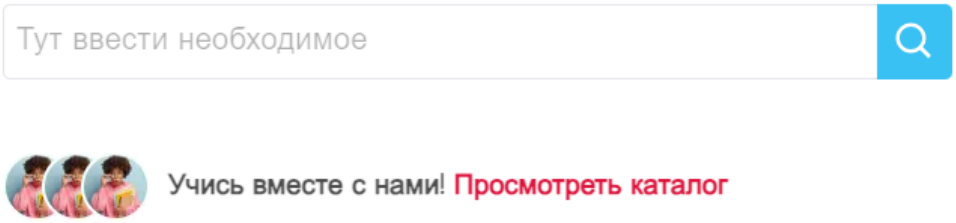


Рисунок 5.15 – Фрагмент страницы с поиском

Также для удобства в каталоге существует сортировка курсов по категориям. Для сортировки на странице Курсы пользователю необходимо выбрать его интересующую и нажать на соответствующую кнопку. Пример приведен на рисунке 5.16.

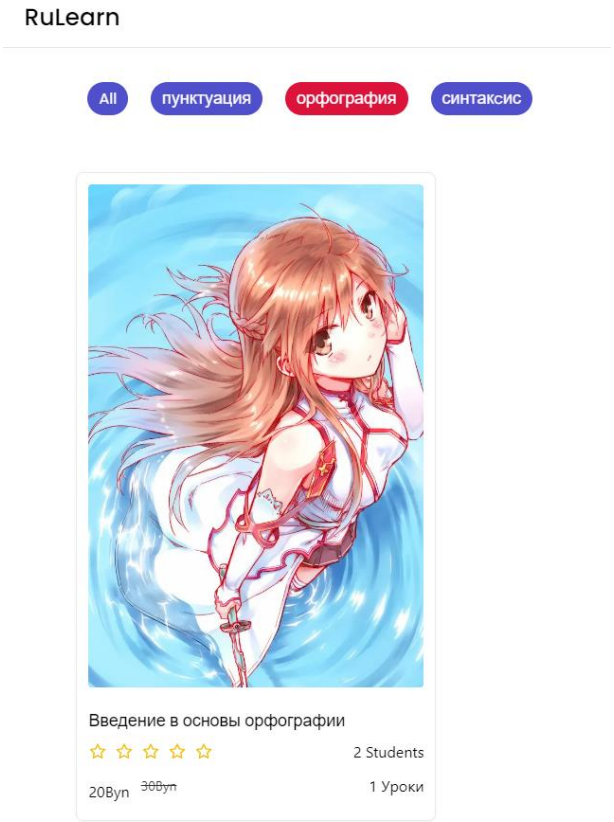


Рисунок 5.16 – Сортировка курсов в каталоге

В приложении также предусмотрена роль администратора. Все действия, описанные выше, входят в его возможности, но также присутствуют и личные.

После входа, администратор может зайти на страницу личного профиля, где будет дополнительная вкладка Admin DashBoard. Пример боковой панели в профиле администратора представлен на рисунке 5.17.

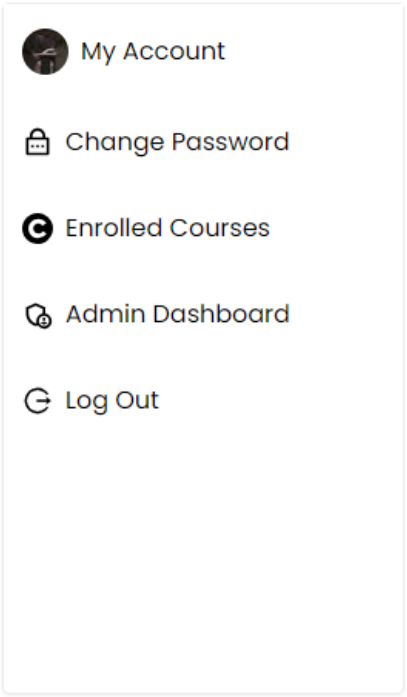


Рисунок 5.17 – Боковая панель в профиле администратора

У администратора есть возможность управления приложением через свою личную панель, которая открывается при переходе по вкладке Admin Dashboard. Результат перехода представлен на рисунке 5.18.



Рисунок 5.18 – Панель администратора

При переходе во вкладку Users имеется возможность просмотреть и удалить список всех пользователей, во вкладке Invoices расположена информация о покупках курсов. Вкладка Manage Team предназначена для управления командой,

имеется возможность изменить роль у пользователя, сделав его администратором и наоборот.

Основным функционалом администратора является создание курса. Получить доступ к форме для создания курса можно перейдя во вкладку Create Course. Результат перехода изображен на рисунке 5.19.

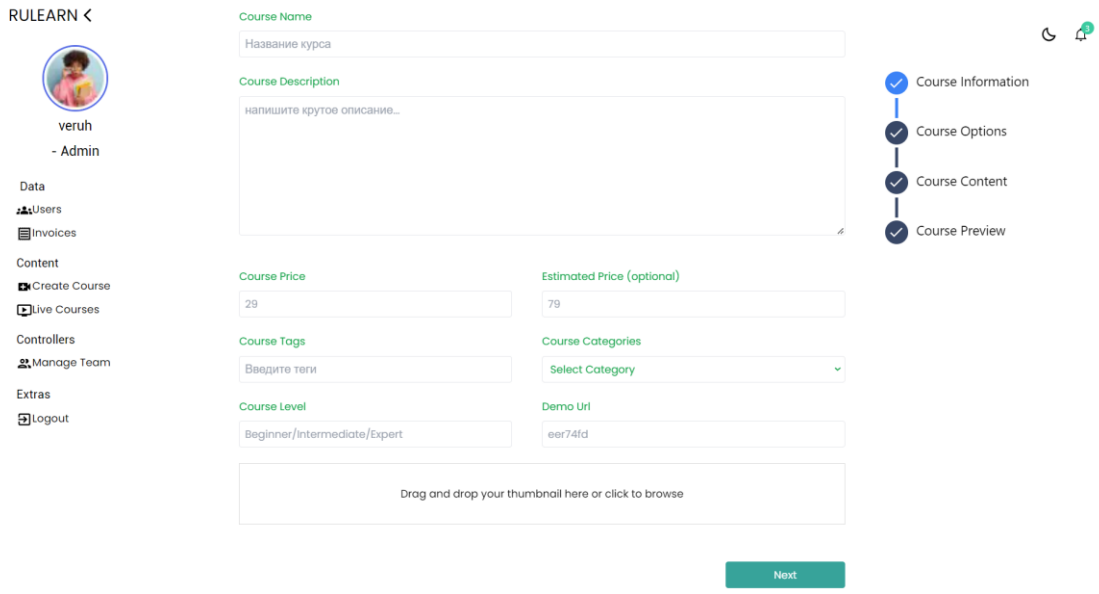


Рисунок 5.19 – Форма создания курса

Перейдя во вкладку Live Course можно просмотреть информацию о существующих курсах, а также изменить или удалить выбранный. Результат перехода во вкладку показан на рисунке 5.20



Рисунок 5.20 – Вкладка Live Courses

Таким образом, было разработано краткое руководство для использования web-приложения, позволяющее быстрее разобраться новому пользователю с внешним интерфейсом и функционалом приложения.

Заключение

В рамках данного курсового проекта было разработано web-приложение «Онлайн-курсы по русскому языку» с использованием серверного фреймворка Express.js, клиентского фреймворка Next.js и базы данных MongoDB.

Перед началом разработки проведен аналитический обзор существующих прототипов приложений аналогичной тематики, а также определены функциональные возможности разрабатываемого приложения.

В процессе разработки серверной части было реализовано API для взаимодействия с базой данных MongoDB, а также была применена стратегия аутентификации с использованием JWT для обеспечения безопасности данных.

Разработанное приложение успешно реализует следующие требования:

– Регистрация и авторизация пользователей;

– Управление курсами, включая добавление, удаление и редактирование теоретических материалов;

– Просмотр каталога курсов и запись на выбранные курсы;

– Прохождение тестов для закрепления знаний;

– Просмотр личной статистики пользователя;

– Поиск курсов по различным критериям.

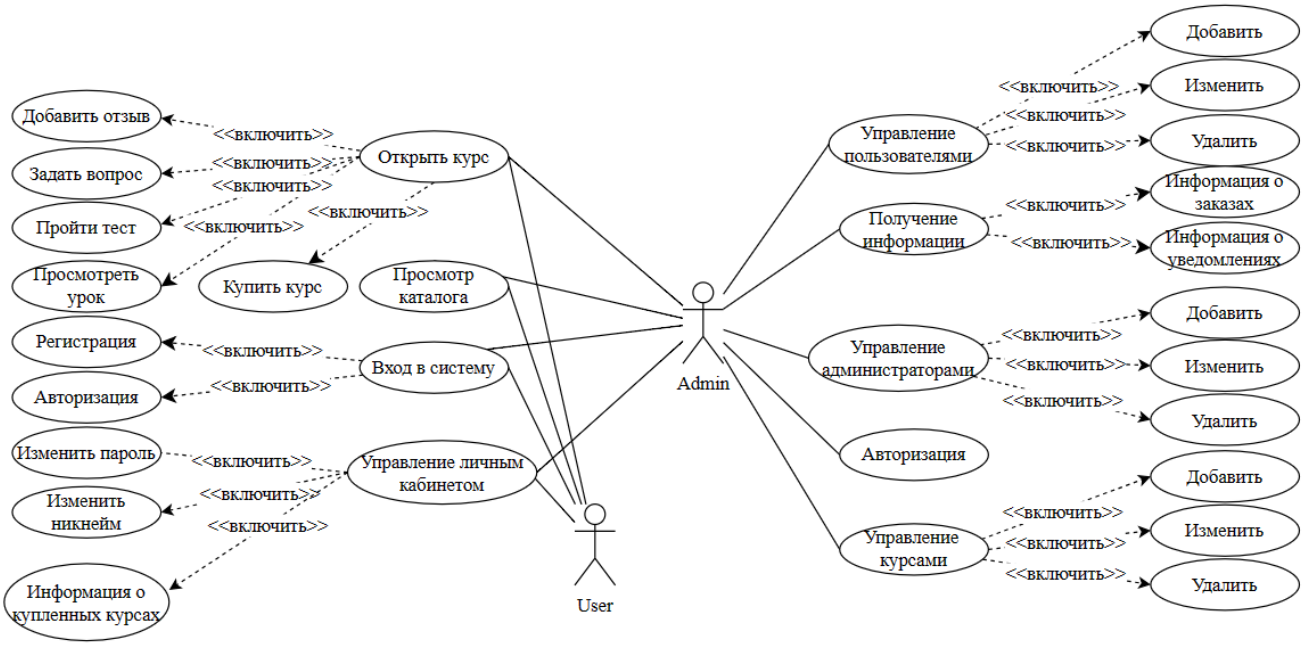
Для обеспечения безопасности передачи данных между клиентом и сервером был создан SSL-сертификат. Важным этапом разработки было тщательное тестирование приложения, в результате которого были выявлены и исправлены ошибки и недочеты.

В целом, разработанное веб-приложение представляет собой эффективный инструмент для обучения русскому языку онлайн, способствующий улучшению процессов обучения и повышению удовлетворенности пользователей.

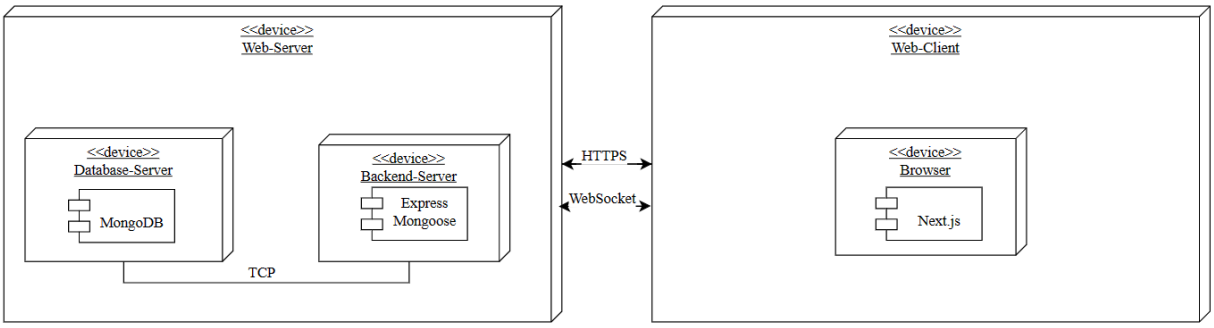
Список используемых источников

1. Документация по фреймворку Express.js [Электронный ресурс] / Режим доступа – URL: https://expressjs.com/ – Дата доступа: 20.03.2024.
2. Документация по фреймворку Next.js [Электронный ресурс] / Режим доступа – URL: https://nextjs.org/ – Дата доступа: 27.03.2024.
3. Официальная документация по MongoDB [Электронный ресурс] / Режим доступа – URL: https://docs.mongodb.com/ – Дата доступа: 30.03.2024.
4. Документация по API Google OAuth 2.0 [Электронный ресурс] / Режим доступа – URL: https://developers.google.com/identity/protocols/oauth2 - Дата доступа: 04.04.2024.
5. Документация по API Nodemailer для отправки писем [Электронный ресурс] / Режим доступа – URL: https://nodemailer.com/about/ - Дата доступа: 06.04.2024.
6. Документация по API Stripe [Электронный ресурс] / Режим доступа – URL: https://stripe.com/docs - Дата доступа: 10.04.2024.

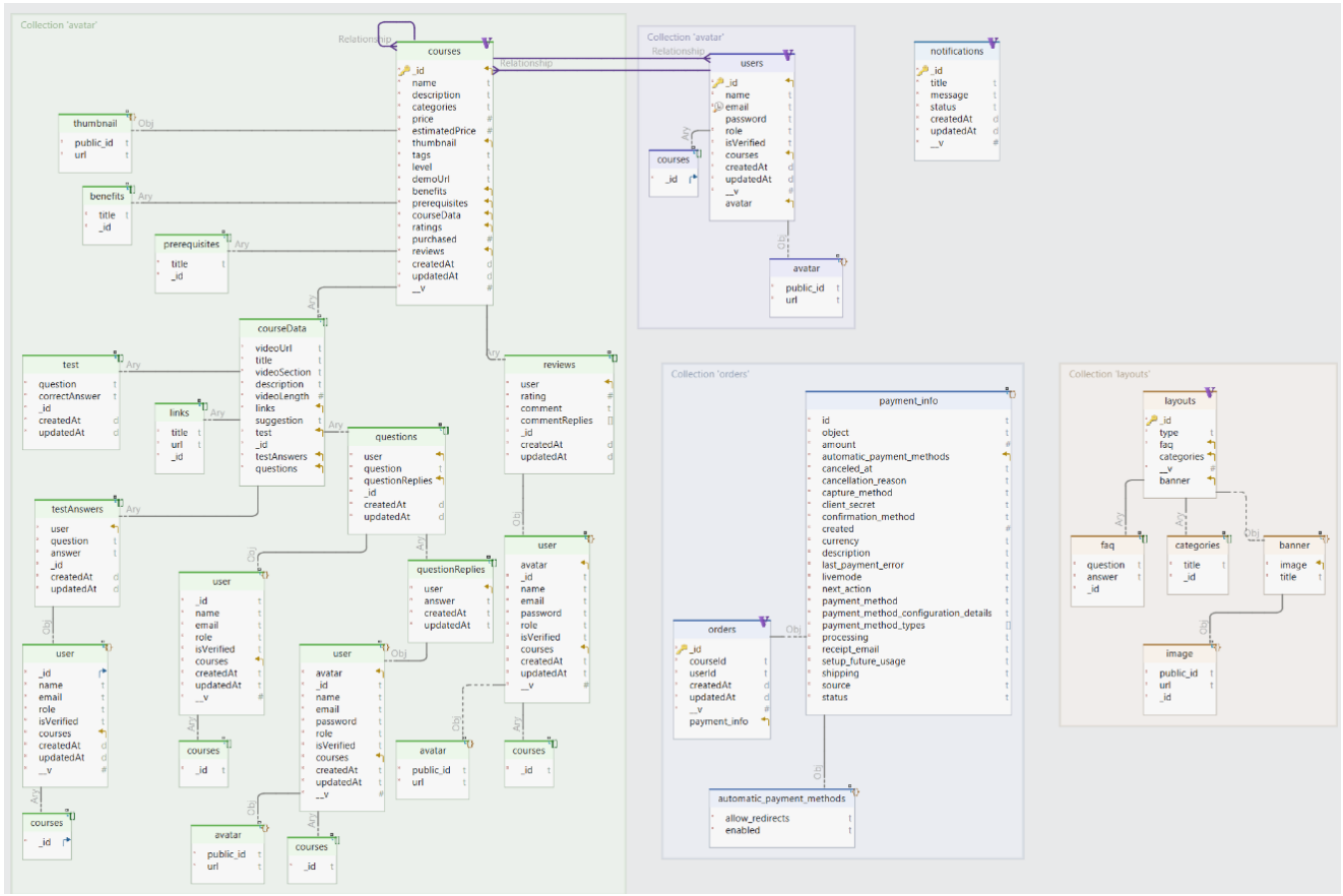
ПРИЛОЖЕНИЕ А



ПРИЛОЖЕНИЕ Б



ПРИЛОЖЕНИЕ В



ПРИЛОЖЕНИЕ Г

```
import mongoose, { Document, IfEquals, Model, Schema } from
"mongoose";
import { IUser } from "./user.model";
    export interface IComment extends Document {
      user: IUser;
      question: string;
      questionReplies: IComment[];}
    interface IReview extends Document {
      user: IUser;
      rating?: number;
      comment: string;
      commentReplies?: IReview[];}
    interface ILink extends Document {
      title: string;
      url: string;}
    interface IAnswer extends Document {
      user: IUser; //
      question: string; // Идентификатор вопроса
      answer: string; // Ответ пользователя}
    // Модель для вопросов и правильных ответов
    interface IQuestion extends Document {
      question: string; // Текст вопроса
      correctAnswer: string; // Правильный ответ}
      interface ICourseData extends Document {
        title: string;
        description: string;
        videoUrl: string;
        videoThumbnail: object;
        videoSection: string;
        videoLength: number;
        videoPlayer: string;
        links: ILink[];
        suggestion: string;
        test: IQuestion[];
        testAnswers: IAnswer[];
        questions: IComment[];}
    export interface ICourse extends Document {
      name: string;
      description: string;
      categories: string;
      price: number;
      estimatedPrice?: number;
      thumbnail: object;
      tags: string;
      level: string;
      demoUrl: string;
      benefits: { title: string }[];
      prerequisites: { title: string }[];
      reviews: IReview[];
      courseData: ICourseData[];
      ratings?: number;
```

```
    purchased: number;}
  const reviewSchema = new Schema<IReview>({
    user: Object,
    rating: {
      type: Number,
      default: 0,},
    comment: String,
    commentReplies: [Object],
  },{timestamps:true});
  const linkSchema = new Schema<ILink>({title: String,url: String,
  });
  const commentSchema = new Schema<IComment>({
    user: Object, question: String, questionReplies: [Object],
  },{timestamps:true});
  const testSchema = new Schema<IQuestion>({
    question: String,correctAnswer: String,
  },{timestamps:true});
  const testAnswersSchema = new Schema<IAnswer>({
    user: Object,question: String,answer: String,
  },{timestamps:true});
  const courseDataSchema = new Schema<ICourseData>({
    videoUrl: String,
    videoThumbnail: Object,
    title: String,
    videoSection: String,
    description: String,
    videoLength: Number,
    videoPlayer: String,
    links: [linkSchema],
    suggestion: String,
    test: [testSchema],
    testAnswers:[testAnswersSchema] ,
    questions: [commentSchema],
  });
  const courseSchema = new Schema<ICourse>({
    name: {type: String,required: true,},
    description: {type: String,required: true,},
    categories:{type:String,required: true,},
    price: {type: Number,required: true,},
    estimatedPrice: {type: Number,},
    thumbnail: {public_id: {type: String,},url: {type: String,},},
    tags:{type: String,required: true,},
    level:{type: String,required: true,},
    demoUrl:{type: String,required: true,},
    benefits: [{title: String}],
    prerequisites: [{title: String}],
    reviews: [reviewSchema],
    courseData: [courseDataSchema],
    ratings:{type: Number,default: 0,},
    purchased:{type: Number,default: 0,},},{timestamps: true});
  const CourseModel: Model<ICourse> = mongoose.model("Course",
courseSchema);

export default CourseModel;
```

ПРИЛОЖЕНИЕ Д

```
import { NextFunction, Request, Response } from "express";
import { CatchAsyncError } from "../middleware/catchAsyncErrors";
import ErrorHandler from "../utils/ErrorHandler";
import { IOrder } from "../models/order.model";
import userModel from "../models/user.model";
import CourseModel, { ICourse } from "../models/course.model";
import path from "path";
import ejs from "ejs";
import sendMail from "../utils/sendMail";
import NotificationModel from "../models/notification.model";
import { getAllOrdersService, newOrder } from
"../services/order.service";
import { redis } from "../utils/redis";
require("dotenv").config();
const stripe = require("stripe")(process.env.STRIPE_SECRET_KEY);
// create order
export const createOrder = CatchAsyncError(
    async (req: Request, res: Response, next: NextFunction) => {
      try {const { courseId, payment_info } = req.body as IOrder;
        if (payment_info) {
          if ("id" in payment_info) {
            const paymentIntentId = payment_info.id;
            const paymentIntent = await
stripe.paymentIntents.retrieve(
              paymentIntentId);
             if (paymentIntent.status !== "succeeded") {
              return next(new ErrorHandler("Payment not authorized!",
400));}}}
        const user = await userModel.findById(req.user?._id);
        const courseExistInUser = user?.courses.some(
          (course: any) => course._id.toString() === courseId);
        if (courseExistInUser) {return next(
new ErrorHandler("You have already purchased this course", 400));}
        const course:ICourse | null = await
CourseModel.findById(courseId);
        if (!course) {return next(new ErrorHandler("Course not
found", 404));}
        const data: any = {courseId: course._id,userId: user?._id,
          payment_info,};
        const mailData = {
          order: {_id: course._id.toString().slice(0, 6),name:
course.name,price: course.price,date: new
Date().toLocaleDateString("ru-ru", {year: "numeric",month: "long",
              day: "numeric",}),},};
        const html = await ejs.renderFile(
          path.join(__dirname, "../mails/order-confirmation.ejs"),
          { order: mailData });
        try {if (user) {
            await sendMail({
              email: user.email,
              subject: "Order Confirmation",
```

```
            template: "order-confirmation.ejs",
            data: mailData,});}} catch (error: any) {
          return next(new ErrorHandler(error.message, 500));}
        user?.courses.push(course?._id);
         await redis.set(req.user?._id, JSON.stringify(user));
        await user?.save();
        await NotificationModel.create({
          user: user?._id,
          title: "New Order",
          message: `You have a new order from ${course?.name}`,});
        course.purchased = course.purchased + 1;
        await course.save();
         newOrder(data, res, next);
      } catch (error: any) {
        return next(new ErrorHandler(error.message, 500));}});
  // get All orders --- only for admin
export const getAllOrders = CatchAsyncError(
  async (req: Request, res: Response, next: NextFunction) => {
    try {
      getAllOrdersService(res);
    } catch (error: any) {
      return next(new ErrorHandler(error.message, 500));}});
//  send stripe publishble key
export const sendStripePublishableKey = CatchAsyncError(
  async (req: Request, res: Response) => {
    res.status(200).json({
      publishablekey: process.env.STRIPE_PUBLISHABLE_KEY,});});
// new payment
export const newPayment = CatchAsyncError(
  async (req: Request, res: Response, next: NextFunction) => {
    try {
      const myPayment = await stripe.paymentIntents.create({
        amount: req.body.amount,
        currency: "USD",
        metadata: {
          company: "RuLearn",
        },
        automatic_payment_methods: {
          enabled: true,
        },
      });
      res.status(201).json({
        success: true,
        client_secret: myPayment.client_secret,
      });
    } catch (error: any) {
      return next(new ErrorHandler(error.message, 500));
    }
  }
);
```